

TECHNICAL IMPLEMENTATION OVERVIEW

AEGIS GCS

Aerospace Extensible Ground Control Station — a modular C++20/Qt6 desktop operator platform for UAS telemetry, mission tooling, plugin-based UI panels, and native map visualization.

C++20

Qt6 Widgets

MAVLink 2

QPluginLoader

OpenStreetMap Tiles

GoogleTest

CMake

Document Version

1.0 — Current Implementation

Generated

2026-05-04

Repository

C:\Users\kutay\Desktop\Projects\aegis-gcs

Git Revision

d6f1298

Table of Contents

- | | |
|--|---|
| 1. Executive Summary | 7. Plugin System |
| 2. Technology Stack | 8. User Interface Shell |
| 3. System Architecture | 9. Map View Implementation |
| 4. Application Runtime and Configuration | 10. Replay and Test Infrastructure |
| 5. Core Services | 11. Build, Deployment, and CI |
| 6. Telemetry and MAVLink Pipeline | 12. Commercial-Grade Readiness Snapshot |

1. Executive Summary

AEGIS GCS is currently implemented as a native desktop Ground Control Station shell. The application is built around a Qt Widgets main window, a typed telemetry bus, a thread-safe vehicle state model, a runtime plugin host, and a MAVLink UDP input/output layer. It supports live/dummy telemetry, log replay parsing, plugin panels, alert visualization, mission editor scaffolding, and a real OpenStreetMap-backed native map view.

Current status: AEGIS is a working modular GCS foundation with core runtime services and multiple plugins. It is not yet a complete certified flight-control product, but its architecture is suitable for iterative hardening toward commercial-grade deployment.

2. Technology Stack

Area	Technology	Current Usage
Language	C++20	Application, plugins, telemetry, tests, and UI are implemented in modern C++.
UI Framework	Qt6 Widgets	Main shell, dock widgets, toolbar/status bar, plugin panels, and native graphics map.
Build System	CMake 3.20+	Cross-platform project structure, static libraries, shared plugin targets, test targets.
Plugin Runtime	Qt <code>QPluginLoader</code>	Runtime discovery/loading of shared-library plugins implementing <code>IPlugin</code> .
Networking	Qt Network	UDP MAVLink I/O via <code>QUdpSocket</code> ; OSM tile downloads via <code>QNetworkAccessManager</code> .
Telemetry Protocol	MAVLink 2 C headers	Message parsing, command packing, heartbeat detection, selected telemetry decoding.
Threading	QThread, Qt queued signals, locks	Dedicated MAVLink I/O thread, cross-thread signal delivery, thread-safe state access.
Testing	GoogleTest + Qt Test	Unit tests for bus/state/ring buffer, parser tests, replay tests, UDP lifecycle test.
CI/CD	GitHub Actions	Linux and Windows build/test workflow scaffold.

3. System Architecture

Qt Widgets Operator Shell — MainWindow, docks, toolbar, status bar

Runtime Plugins — Map View, Telemetry HUD, Alert Console, Mission Editor

Core Services — TelemetryBus, VehicleState, PluginHost

Telemetry Services — MavlinkIO, MavlinkParser, LogReplay

External Interfaces — UDP MAVLink, OpenStreetMap tile server, replay files

The current architecture separates the long-lived application shell from runtime-loaded plugin functionality. The **Application** class owns the core service graph, connects signal/slot pipelines, loads configuration, discovers plugins, and starts optional dummy telemetry.

Repository Structure

```
aegis-gcs/
├── config/           Runtime JSON configuration
├── resources/        Qt resource files
├── src/app/           Application bootstrapper and service wiring
├── src/core/          Bus, state model, plugin host, interfaces, shared types
├── src/plugins/       Runtime-loaded Qt plugin modules
├── src/telemetry/     MAVLink I/O, parser, replay, telemetry types
├── src/ui/            Main window, dock manager, theme, status widgets
├── src/utils/         Logger, ring buffer, thread pool
├── tests/             GoogleTest/Qt tests
└── third_party/mavlink/ MAVLink C headers
```

4. Application Runtime and Configuration

The application loads JSON configuration from `config/aegis.json`, merges it with built-in defaults, validates key fields, and uses the normalized config to drive runtime behavior.

Implemented Config Domains

Domain	Purpose
<code>pluginPaths</code>	Additional directories for plugin discovery.
<code>autostartPlugins</code>	Plugin IDs to load at startup.
<code>telemetry</code>	UDP bind address, bind port, heartbeat timeout.
<code>logging</code>	Minimum log level and log file path.
<code>dummyTelemetry</code>	Offline/demo telemetry generator enablement and interval.
<code>plugins</code>	Per-plugin settings passed to each plugin during initialization.

```
{
  "telemetry": {
    "bindAddress": "0.0.0.0",
    "bindPort": 14550,
    "heartbeatTimeoutMs": 3000
  },
  "dummyTelemetry": { "enabled": true, "intervalMs": 100 },
  "plugins": {
    "aegis.plugins.map_view": {
      "zoom": 15,
      "tileUrlTemplate": "https://tile.openstreetmap.org/%1/%2/%3.png"
    },
    "aegis.plugins.alert_console": {
      "maxItems": 250,
      "showTimestamps": true
    }
  }
}
```

5. Core Services

TelemetryBus

Typed Qt signal bus for hot-path telemetry and system events. It emits attitude, position, battery, heartbeat, alerts, connection state, command responses, and outbound vehicle commands.

VehicleState

Thread-safe canonical model of the active vehicle. Uses `QReadWriteLock` and exposes typed getters/mutators with update signals.

PluginHost

Discovers shared libraries, reads Qt plugin metadata, validates `IPlugin`, initializes plugins with bus/state/config, and manages unload lifecycle.

Shared Types

Defines telemetry and command data structures: `AttitudeData`, `PositionData`, `BatteryData`, `SystemState`, and `VehicleCommand`.

6. Telemetry and MAVLink Pipeline

The live telemetry path receives MAVLink frames over UDP on a dedicated worker thread. Parsed messages are emitted through Qt queued connections to the main-thread parser, which updates `VehicleState` and broadcasts typed bus events.

```
UDP Socket → MavlinkIO → MavlinkParser → VehicleState + TelemetryBus → UI/Plugins
```

Current MAVLink Decoding

MAVLink Message	AEGIS Output
HEARTBEAT	System ID/component, base mode, custom mode, armed state, system status.
ATTITUDE	Roll, pitch, yaw, angular rates.
GLOBAL_POSITION_INT	Latitude, longitude, altitude, relative altitude, velocity, heading.
BATTERY_STATUS	Remaining percentage, voltage, current.
MISSION_CURRENT	Current mission index.
STATUSTEXT	Alert severity mapping and alert console output.

Connection Lifecycle

`MavlinkIO` now separates socket availability from vehicle health. It only emits connected after receiving a MAVLink heartbeat. If no heartbeat arrives within the configured timeout, it emits disconnected. Shutdown stops timers, closes sockets, clears remote state, and waits for the worker thread.

Important design decision: socket bind success is not treated as a vehicle connection. A vehicle is considered alive only after heartbeat evidence.

7. Plugin System

Plugins are shared libraries implementing the `IPlugin` interface. Each plugin receives pointers to the global `TelemetryBus` and `VehicleState`, plus a plugin-specific configuration map.

Plugin	Status	Description
Map View	Implemented	Main central map using native Qt graphics and OpenStreetMap tiles.
Telemetry HUD	Implemented	Displays attitude, battery, and system state.
Alert Console	Implemented	Scrollable severity-colored alert list with configurable history and timestamps.
Mission Editor	Basic	Waypoint table and command-source scaffold; full MAVLink mission protocol is future work.

8. User Interface Shell

The UI shell is a Qt Widgets `QMainWindow` with menu bar, toolbar, status bar, central map area, and right-side docked plugin panels. The map plugin is injected as the central widget. Other plugins are wrapped in `QDockWidget` containers and tabified.

Current UI Features

- Dark operator theme via `ThemeEngine`.
- File menu for opening replay logs.
- Connect menu and toolbar action for UDP MAVLink.
- Status bar connection indicator.
- Vehicle armed/mode display derived from heartbeat state.
- Plugin injection and basic dock/tab management.

9. Map View Implementation

The current map uses Qt native graphics rather than an embedded browser. It converts latitude/longitude to Web Mercator pixel coordinates, downloads OpenStreetMap tiles, places them in a `QGraphicsScene`, and overlays vehicle position, heading, and track history.

Feature	Implementation
Tile provider	Configurable URL template, defaulting to <code>https://tile.openstreetmap.org/%1/%2/%3.png</code> .
Projection	Web Mercator tile coordinate formula.
Networking	<code>QNetworkAccessManager</code> and <code>QNetworkReply</code> .
Rendering	<code>QGraphicsView</code> , <code>QGraphicsScene</code> , pixmap tile items, path overlay, marker overlay.
Zoom	Changes OSM tile zoom level and refreshes tile set.

Operational note: OSM tile usage requires respecting OpenStreetMap tile policy. A commercial deployment should add tile caching, provider attribution, rate limiting, and preferably a dedicated commercial tile provider or offline MBTiles support.

10. Replay and Test Infrastructure

The replay service reads binary MAVLink frames from log files and emits decoded messages into the normal parser path. This allows replayed telemetry to update the same state/bus/UI pipeline as live UDP telemetry.

Current Automated Tests

Test Target	Coverage
<code>test_telemetry_bus</code>	Signal delivery, topic pub/sub, outbound commands, command responses.
<code>test_vehicle_state</code>	Thread-safe state update/getter behavior.
<code>test_ring_buffer</code>	Fixed-size telemetry history behavior.
<code>test_parsers</code>	HEARTBEAT, ATTITUDE, GLOBAL_POSITION_INT, BATTERY_STATUS.
<code>test_log_replay</code>	Replay step and playback completion/progress.
<code>test_mavlink_io</code>	UDP heartbeat connection and timeout disconnection.

11. Build, Deployment, and CI

The project builds with CMake and Qt6. Core libraries are static targets; plugins are shared libraries deployed under the runtime `plugins/` directory. On Windows, the build integrates `windeployqt` for Qt runtime DLL deployment.

```
cmake -B build -S . -DCMAKE_PREFIX_PATH="C:/Qt/6.8.2/msvc2022_64" -DAEGIS_BUILD_TESTS=ON
cmake --build build --config Release --parallel
ctest --test-dir build --output-on-failure -C Release
```

GitHub Actions currently defines Linux and Windows build/test jobs using Qt installation action, CMake, and CTest.

12. Commercial-Grade Readiness Snapshot

Area	Current State	Commercial Next Step
Architecture	Modular and plugin-based.	Stabilize plugin SDK/ABI and add service registry.
Telemetry	Selected MAVLink messages decoded.	Complete mission, parameter, command ACK, signing, and multi-vehicle support.
Map	Real OSM online tile layer.	Add offline cache, provider attribution, commercial provider, and geofence editing.
Safety	Heartbeat timeout and alerts scaffold.	Add full failsafe rules, command confirmation, and audit logging.
Testing	Growing unit/integration test suite.	Add fuzzing, UI smoke tests, sanitizers, replay regression corpus, and hardware-in-loop tests.
Deployment	Build and Windows runtime deployment scaffold.	Add installer, updater, crash reporter, license management, and diagnostics export.

This document describes the current implementation as of git revision d6f1298. It is intended as a technical overview for development, planning, onboarding, and investor/customer-facing engineering discussions.